Project Documentation: 3-Bit Binary Counters and T-Flip-Flop

Digital Design Lab

Project Overview

This lab explores different modeling paradigms in Verilog HDL for designing digital counters[cite: 29, 30, 31]. The modules implement a **3-Bit Binary Up-Counter** using both behavioral addition and explicit Finite State Machine (FSM) representations, along with a foundational **T-Flip-Flop** building block[cite: 29, 30, 31]. All modules share an asynchronous active-low reset control (`reset_n`)[cite: 29, 30, 31].

Specifications & Functionality

- **Behavioral Counter (`counter_behavioral`)**: Increments a 3-bit register (`count`) on every rising edge of the clock using direct arithmetic addition (`count + 1'b1`)[cite: 29].
- **FSM Counter (`counter_fsm`)**: Implements counter states explicitly from `S0` (`3'b000`) through `S7` (`3'b111`) using a 3-always-block FSM pattern, mapping state directly to output[cite: 30].
- **T Flip-Flop (`t_ff`)**: Toggles its state bit (`q`) on rising clock edges when enable line `t` is high; otherwise holds its previous state[cite: 31].
- **Asynchronous Reset**: Active-low signal `reset_n` immediately clears counter values/flip-flop outputs to `0` on its falling edge[cite: 29, 30, 31].

Port Summary

Port Name	Direction	Type	Width	Description
`clk`	Input	`wire`	1	Clock input signal[cite: 29, 30, 31]
`reset_n`	Input	`wire`	1	Asynchronous active-low reset line[cite: 29, 30, 31]
`t`	Input	`wire`	1	Toggle control line (T-FF only)[cite: 31]
`count`	Output	`reg`	3	3-bit binary counter value[cite: 29, 30]
`q`	Output	`reg`	1	Output state bit (T-FF only)[cite: 31]

Table 1: 3-Bit Counter and T-FF Port Interfaces

—

Verilog Source Code

Behavioral Counter (`counter_behavioral.v`)

```
module counter_behavioral (
    input  wire       clk,
    input  wire       reset_n,
    output reg  [2:0] count
```

```
);

    // Increment count on each rising clock edge or clear on reset
    always @(posedge clk or negedge reset_n) begin
        if (!reset_n)
            count <= 3'b000;
        else
            count <= count + 1'b1;
    end

endmodule
```

—

FSM-Based Counter (counter_fsm.v)

```
module counter_fsm (
    input  wire       clk,
    input  wire       reset_n,
    output reg  [2:0] count
);

    localparam S0 = 3'b000,
               S1 = 3'b001,
               S2 = 3'b010,
               S3 = 3'b011,
               S4 = 3'b100,
               S5 = 3'b101,
               S6 = 3'b110,
               S7 = 3'b111;

    reg [2:0] current_state, next_state;

    // State Register with Asynchronous Active-Low Reset
    always @(posedge clk or negedge reset_n) begin
        if (!reset_n)
            current_state <= S0;
        else
            current_state <= next_state;
    end

    // Next State Logic
    always @(*) begin
        case (current_state)
            S0: next_state = S1;
            S1: next_state = S2;
            S2: next_state = S3;
            S3: next_state = S4;
            S4: next_state = S5;
            S5: next_state = S6;
            S6: next_state = S7;
            S7: next_state = S0;
            default: next_state = S0;
        endcase
    end

    // Output Mapping
    always @(*) begin
```

```
        count = current_state;
    end

endmodule
```

—

T Flip-Flop Building Block (t_ff.v)

```
module t_ff (
    input  wire clk,
    input  wire reset_n,
    input  wire t,
    output reg  q
);

    // Toggle logic on active clock edge
    always @(posedge clk or negedge reset_n) begin
        if (!reset_n)
            q <= 1'b0;
        else if (t)
            q <= ~q;
    end

endmodule
```

—

Testbench Module (tb_counters.v)

```
`timescale 1ns / 1ps

module tb_counters;

    reg clk;
    reg reset_n;

    wire [2:0] count_beh;
    wire [2:0] count_fsm;

    // Instantiate Behavioral Counter
    counter_behavioral uut_beh (
        .clk(clk),
        .reset_n(reset_n),
        .count(count_beh)
    );

    // Instantiate FSM Counter
    counter_fsm uut_fsm (
        .clk(clk),
        .reset_n(reset_n),
        .count(count_fsm)
    );

    // Clock Generation (10ns Period)
    always #5 clk = ~clk;
```

```
    initial begin
        clk     = 0;
        reset_n = 0;

        #12;
        reset_n = 1;

        #100;
        reset_n = 0;

        #10;
        reset_n = 1;

        #40;
        $finish;
    end

    // Monitor Output
    initial begin
        $monitor("Time = %0t | reset_n = %b | Behavioral Count = %b (%0
           d) | FSM Count = %b (%0d)",
                 $time, reset_n, count_beh, count_beh, count_fsm,
                    count_fsm);
    end

endmodule
```

—

Simulation & Verification

Execution Command

To execute batch simulation using Cadence Xcelium or ModelSim:

```
xrun counter_behavioral.v counter_fsm.v t_ff.v tb_counters.v
```

Expected Simulation Output

The simulation trace demonstrates identical functionality across both behavioral and FSM implementations, including asynchronous reset response:

```
Time = 0 | reset_n = 0 | Behavioral Count = 000 (0) | FSM Count = 000
    (0)
Time = 12 | reset_n = 1 | Behavioral Count = 000 (0) | FSM Count = 000
    (0)
Time = 15 | reset_n = 1 | Behavioral Count = 001 (1) | FSM Count = 001
    (1)
Time = 25 | reset_n = 1 | Behavioral Count = 010 (2) | FSM Count = 010
    (2)
Time = 35 | reset_n = 1 | Behavioral Count = 011 (3) | FSM Count = 011
    (3)
Time = 45 | reset_n = 1 | Behavioral Count = 100 (4) | FSM Count = 100
    (4)
Time = 55 | reset_n = 1 | Behavioral Count = 101 (5) | FSM Count = 101
    (5)
Time = 65 | reset_n = 1 | Behavioral Count = 110 (6) | FSM Count = 110
    (6)
```

```
Time = 75 | reset_n = 1 | Behavioral Count = 111 (7) | FSM Count = 111
    (7)
Time = 85 | reset_n = 1 | Behavioral Count = 000 (0) | FSM Count = 000
    (0)
Time = 112 | reset_n = 0 | Behavioral Count = 000 (0) | FSM Count = 000
     (0)
```